

API REST — Sistema de Gestión de Biblioteca

Documento Técnico · Proyecto Integrador UT3+UT4+UT5

Spring Boot 3.5.6 · JPA · H2 · Java 21

Tecnología:	Spring Boot 3.5.6 · JPA · H2 · Java 21
Versión API:	/api/v1
Base de datos:	H2 en memoria (dev) — compatible MySQL/PostgreSQL
Tests:	23 tests pasando — BUILD SUCCESS

1. Arquitectura por Capas

El proyecto sigue una arquitectura en tres capas desacopladas. Cada capa tiene una responsabilidad única y se comunica solo con la capa adyacente:

Capa	Clase	Responsabilidad
Presentación	Controller (@RestController)	Recibe HTTP, valida con @Valid, devuelve Resource
Negocio	Service (@Service)	Aplica reglas de negocio, coordina repositorios, usa Mapper
Persistencia	Repository (JpaRepository)	Acceso a BD, consultas derivadas, Optional en búsquedas

Flujo de una petición

Cliente → Controller (recibe DTO, valida) → Service (lógica de negocio) → Repository (BD) → Mapper (entidad a Resource) → Cliente

2. Modelo de Entidades y Relaciones

El dominio modela un sistema de biblioteca con 4 entidades principales:

Entidad	Atributos principales	Relaciones
Autor	id, nombre, apellido, nacionalidad, fechaNacimiento	1 Autor → N Libros (@OneToMany)
Libro	id, titulo, isbn, anioPublicacion, stockTotal, stockDisponible, genero	N Libros → 1 Autor (@ManyToOne) 1 Libro → N Prestamos (@OneToMany)
Usuario	id, nombre, apellido, email, telefono, activo	1 Usuario → N Prestamos (@OneToMany)
Prestamo	id, fechaPrestamo, fechaDevolucionPrevista, fechaDevolucionReal, estado	N Prestamos → 1 Usuario (@ManyToOne) N Prestamos → 1 Libro (@ManyToOne)

3. Decisiones de Diseño

3.1 DTOs de entrada y Resources de salida

Se separaron los objetos de entrada (DTOs) de los de salida (Resources) para cada entidad. Los DTOs llevan las anotaciones de validación (@NotBlank, @Email, @Positive, @Future). Los Resources pueden incluir datos calculados que no existen directamente en la entidad, como nombreAutor en LibroResource o prestamosActivos en UsuarioResource.

3.2 Mappers manuales

Se optó por mappers manuales en lugar de MapStruct para mayor control. Cada mapper tiene tres métodos: toEntity() para convertir un CreatedTO a entidad, toResource() para convertir una entidad a Resource, y updateEntityFromDTO() para aplicar cambios parciales (PATCH) solo si el campo no es null.

3.3 Flujo de negocio no trivial — Préstamo

El servicio de préstamos implementa tres reglas de negocio en el método create():

- Regla 1: El usuario debe estar activo (activo = true) para poder tomar prestado un libro.
- Regla 2: El libro debe tener stock disponible (stockDisponible > 0).
- Regla 3: El usuario no puede tener ya ese mismo libro en préstamo activo.

Al crear un préstamo el stock disponible se decrementa en 1. Al devolver, se incrementa en 1. El estado puede ser ACTIVO, DEVUELTO o VENCIDO.

3.4 Versionado de la API

Todos los endpoints están bajo el prefijo /api/v1 definido en @RequestMapping de cada controller. Esto permite futuras versiones (/api/v2) sin romper compatibilidad.

4. Gestión de Errores

Se implementó un manejador global con @RestControllerAdvice que centraliza todas las respuestas de error en un formato JSON consistente:

Excepción	HTTP	Cuándo se lanza
RecursoNoEncontradoException	404	findById() no encuentra el recurso en BD
ConflictoException	409	ISBN duplicado, email duplicado, autor duplicado
ReglaNegocioException	400	Usuario inactivo, sin stock, préstamo duplicado
MethodArgumentNotValidException	400	Falla @Valid en DTO de entrada
Exception (genérica)	500	Cualquier error no controlado

Formato de respuesta de error

```
{
  "timestamp": "2026-05-02T16:30:00",
  "status": 404,
  "error": "Not Found",
  "mensaje": "Libro no encontrado con id: 99"
}
```

5. Ejemplos JSON

5.1 POST /api/v1/prestamos — Entrada

```
{
  "usuarioId": 1,
  "libroId": 2,
  "fechaDevolucionPrevista": "2026-06-01"
}
```

5.2 POST /api/v1/prestamos — Respuesta 201

```
{
  "id": 1,
  "usuarioId": 1,
  "nombreUsuario": "Ana Martinez",
  "tituloLibro": "El amor en los tiempos del colera",
  "fechaPrestamo": "2026-05-02",
  "estado": "ACTIVO"
}
```

5.3 Validación fallida — 400 Bad Request

```
{
  "status": 400,
  "error": "Error de validacion",
  "errores": { "email": "El email es obligatorio" }
}
```

6. Resumen de Endpoints (Solo algunos)

Método	Endpoint	Descripción	HTTP
GET	/api/v1/autores	Listar todos los autores	200
GET	/api/v1/autores/{id}	Obtener autor por id	200 / 404
POST	/api/v1/autores	Crear autor	201 / 409
PUT	/api/v1/autores/{id}	Actualizar autor completo	200 / 404
PATCH	/api/v1/autores/{id}	Actualizar autor parcial	200 / 404
DELETE	/api/v1/autores/{id}	Eliminar autor	204 / 404
GET	/api/v1/libros/disponibles	Libros con stock > 0	200
GET	/api/v1/libros/genero/{genero}	Filtrar libros por género	200
POST	/api/v1/prestamos	Crear préstamo (flujo negocio)	201 / 400
PATCH	/api/v1/prestamos/{id}/devolver	Devolver libro	200 / 400
GET	/api/v1/prestamos/vencidos	Préstamos vencidos activos	200
GET	/api/v1/usuarios/activos	Usuarios activos	200